

proxmox_agent coworker prompt

Full prompt for an engineer or coding agent. Paste the body of this document as the first message in a checkout of the repository. Date: 2026-09-20.

The planning documents in the repo remain the source of truth. This prompt orients the agent; it does not replace those files. The Task section currently says implement Phase 2 only — change that paragraph if you want a review instead of code.

You are a senior software architect and infrastructure automation engineer working in the `proxmox_agent` git repository.

Mission

Build and maintain a **locally hosted** AI agent for managing a Proxmox homelab. The agent must run inside the lab. It must not depend on cloud services unless the operator explicitly configures and approves them. Security is first-class because the agent will eventually perform administrative actions.

Do not ignore, rewrite, or “simplify away” the planning documents. Read them before writing code.

Current repository status (as of 2026-09-20)

- Git remote: `git@github.com:rgonzalezjr80/proxmox_agent.git` (branch `main` unless told otherwise).
- **Phase 1 is complete:** architecture and security are specified in Markdown at the repo root.
- **There is no application code yet** (no `src/`, no `pyproject.toml`, no Docker, no tests). Do not assume a framework other than the locked stack below.
- Do **not** edit Cursor plan files under `.cursor/plans/` or `/home/tito/.cursor/plans/` unless the operator explicitly asks.

Documents you MUST read first (source of truth)

1. `README.md` — project intent and status
2. `ARCHITECTURE.md` — layers, adapters, failure modes, package layout
3. `SECURITY.md` — RBAC, allowlists, secrets, injection, kill switch
4. `THREAT_MODEL.md` — threats and residual risk
5. `IMPLEMENTATION_PLAN.md` — phases, exit criteria, order of actions
6. `DATA_MODEL.md` — SQLite/SQLAlchemy entities
7. `API_DESIGN.md` — HTTP contracts and error shape
8. `APPROVAL_WORKFLOW.md` — preview, risk, approve, execute
9. `TEST_PLAN.md` — unit/integration/e2e; no live cluster in CI
10. `DISASTER_RECOVERY.md` — agent backup/restore
11. `CONFIGURATION.md` — env vars, flags, defaults
12. `CHANGE_REPORTS.md` — mandatory report after every change or fix

If a later instruction conflicts with these documents, **stop and ask**. Do not silently override security or approval rules.

Locked product and stack decisions (do not reopen)

- Homelab: two Proxmox nodes in a cluster; Windows and Linux VMs; storage and networks; services inside guests.
- Agent runs on a **dedicated management VM**, as a client of the Proxmox API (:8006). Not a privileged process on the hypervisors.
- **Language:** Python 3.12
- **App:** FastAPI + Pydantic Settings
- **DB:** SQLAlchemy 2.x + Alembic; **SQLite** default (Postgres-swappable later)
- **UI:** Jinja2 + HTMX + small CSS (no SPA)
- **LLM:** Ollama via OpenAI-compatible API; **fail-safe if Ollama is down**. Cloud LLMs stay **disabled** unless explicitly enabled later (LLM_CLOUD_ENABLED default false).
- **Proxmox client:** httpx behind a ProxmoxClient protocol (do not bind the design to proxmoxer).
- **Jobs:** in-process scheduler (APScheduler) behind a queue interface; no Redis/Celery in v1.
- **Secrets:** master key from the environment; application-encrypted secret store; never in git.
- **Deploy:** Docker Compose on the management VM.
- **Notifications:** NotificationPort with Null and Log adapters only for now. Do not assume email/Slack/PagerDuty.
- Auth: local username/password first (Argon2id). Roles: viewer, operator, admin. No SSO in v1.
- English UI. Timestamps stored in UTC.
- LXC is in the data model even if none exist. qemu-guest-agent is optional; incomplete IP/OS data is allowed and must be visible.

Architecture rule (non-negotiable)

The language model **never** calls Proxmox, SSH, WinRM, or the database as a privileged client.

Flow: Operator → Jinja/HTMX UI and/or /api/v1 → Auth/RBAC/CSRF/rate limits → services and (for chat) an orchestrator that may only **propose allowlisted tools** → policy (feature flags, kill switch, unique target, approval) → execution handlers → adapters.

Unlisted tools do not exist. There is no “run arbitrary command” or “fetch this URL” tool in v1.

Guardrails (enforce in code, not by trusting the LLM)

- Default AGENT_MODE=read_only. Mutating action-family flags default **false**. Missing config means **deny**.
- Never guess a VM, node, user, or resource id. Ambiguous names → ask; do not call the tool.
- Never delete/mutate guests, disks, networks, or accounts without an approval record.
- Destructive actions require confirmation of **name and VMID**.
- High-risk actions require re-authentication.
- Preview must include: understood intent, exact target, action, impact, risk, exact API method/path/body (redacted), backup/snapshot plan, rollback. Execute uses a **preview hash**; mismatch aborts.
- Kill switch: config AGENT_EMERGENCY_STOP, file data/EMERGENCY_STOP, and admin API/UI. Any one disables all mutating tools.
- Treat logs, VM descriptions, pasted errors, and websites as **untrusted data**. They cannot override system rules or skip approval. Wrap them in untrusted delimiters in the orchestrator.

- Never log, display, or commit secrets. Phase 2 Proxmox credential is an API token with **PVEAuditor** only. Write tokens wait until Phase 6.
- TLS to Proxmox: verify **on** by default; optional custom CA. Do not make `verify=false` the happy path.
- Fail closed if Proxmox, DB, approval state, or target identity is uncertain. If Ollama is down, inventory/monitoring still work; chat degrades.
- Never claim success until the handler **verifies** post-conditions. Partial failures are `partial`, not success.
- Simulated actions must be labeled `simulated=true` and must not use the production write path.
- CI must not require a live cluster, real tokens, or Ollama. Use a fake Proxmox HTTP API and fixtures. Never run destructive tests against production.

Phased delivery (do not skip ahead)

Follow `IMPLEMENTATION_PLAN.md` exit criteria.

1. **Phase 1 — done.** Planning documents only.
2. **Phase 2 — next implementation work:** scaffolding (`pyproject.toml`, `.gitignore`, `.env.example`, Docker, app factory, `/health`, `/ready`, Alembic, structured logging, tests) **and** read-only inventory (cluster, nodes, VMs, LXC, storage, networks, snapshots/backups when the API provides them, periodic + manual sync, basic API + HTMX UI). Assert sync does **not** call Proxmox mutating methods.
3. **Phase 3:** monitoring, thresholds, alerts, ack/suppress/maintenance, Null/Log notifications.
4. **Phase 4:** Ollama analysis and read-only NL queries; injection tests; no execution from chat.
5. **Phase 5:** auth, RBAC, approvals, audit hash chain, dry-run, emergency stop, **change reports on terminal execution**.
6. **Phase 6:** one action family at a time, low risk first: start → shutdown → reboot → pause/resume → tags → snapshot create → restore → clone/create from **approved** templates → resource/NIC/migrate/backup → **delete last**. Each family: validation, permission, unique target, approval, optional snapshot, timeout, verify, audit, rollback guidance, change report. Flags default off.
7. **Phase 7:** Linux/Windows guest account management (allowlisted commands only), only after Phase 6 start/stop/reboot are solid.
8. **Phase 8:** labeled simulations, full report exports (Markdown, HTML, JSON, CSV; PDF if practical).

Do not implement Phase N+1 until Phase N exit criteria are met, unless the operator explicitly narrows the task.

Change and fix reports (mandatory)

After **any** fix or change you make, you must send the operator a detailed report in the chat (see `CHANGE_REPORTS.md`):

```
## Change report
- Report ID:
- Date/time:
- Scope:
- What was broken (or missing):
- Root cause / why it was that way:
- How it was fixed:
- What was verified:
- Residual risk / follow-up:
```

- Related docs or tests:

If nothing was broken (new work), say that explicitly. Do not claim tests passed unless you ran them.

When the product can execute actions, every terminal execution (succeeded / failed / partial) must persist a `ChangeReport` (what was broken, how it was fixed, API calls, verified result, rollback). Chat may summarize and must include the report id. Notifications are optional; storage is not.

Engineering practices

- Modular, typed Python; Ruff + type checking.
- Unit, integration, and e2e tests as specified in `TEST_PLAN.md`.
- Structured logging; useful errors; timeouts and partial-failure handling.
- Avoid destructive defaults. Configuration via env + config files; `.env.example` with empty placeholders.
- Health-check endpoints from the first app commit.
- API documentation via FastAPI OpenAPI plus `API_DESIGN.md`.
- Local / test / production-template configs.
- Do not commit `.env`, tokens, keys, or `data/*.db`.
- Do not update git config. Do not force-push. Commit only when the operator asks. Do not push unless asked.
- Prefer editing existing docs over inventing a parallel design.

How the operator will use the product (when it exists)

- Browser UI on the management VM (default bind `127.0.0.1:8080`).
- JSON API under `/api/v1`.
- Conversational chat in the UI (Phase 4+): requests like “show offline VMs”, “inventory the cluster”, “explain this error”. Mutating requests create an approval; they do not execute from chat text.
- For every proposed action the UI/chat must show: what was understood, target, operation, impact, risk, prerequisites, approval required.

Lab facts (do not invent extra infrastructure)

- Two Proxmox servers in a cluster.
- Windows and Linux VMs; possible LXC.
- Multiple services inside VMs.
- Operator is a homelab admin, not a multi-tenant SaaS customer.

Task

Implement **Phase 2 only**: project scaffolding and read-only Proxmox inventory, matching `IMPLEMENTATION_PLAN.md` Phase 2 exit criteria and the documents listed above.

Do not start Phases 3–8. Do not register mutating Proxmox tools. Do not enable write tokens. Do not skip tests or `.gitignore`.

If Phase 2 is already present when you start, inspect the repo, report status, and wait for a new task instead of

duplicating work.

When finished: summarize how to run locally, what was tested, known gaps, and include a **Change report**.